

Schreier Sims

Schreier–Sims 算法

前置知识：[抽象代数基本概念](#)、[群论](#)、[置换与排列](#)

引入

Schreier–Sims 算法 是计算群论 (computational group theory) 的一种算法，以数学家 Otto Schreier 和 Charles Sims 的名字命名。该算法能够在多项式时间内解决诸如找到有限置换群的阶数、查看给定置换是否包含在所给群中等许多问题。Schreier–Sims 算法最早由 Sims 在 1970 年基于 Schreier 引理引入。在 1981 年¹，Donald Knuth 进一步改进了该算法的运行时间。后来，该算法又发展出来一种更快的随机化版本。计算机代数系统（例如 GAP 和 Magma）通常使用该算法的高度优化过的 Monte Carlo 版本²。

▼ 记号

本文依照计算群论文献的惯例，将群作用记作右作用，这意味着置换的复合由左向右进行。本文涉及的群作用都可以视为置换作用，尽管部分算法对于更广泛的群作用也成立。相应地，群作用的集合默认为 G ，其中的元素则称为点。置换作用在点 α 上得到的结果记作 αg ，有时也称置换 g 将点 α 移动到点 αg 。最后，置换群 G 作用下，点 α 的轨道记作 αG ，它的稳定化子则记作 $\text{Stab}_G(\alpha)$ 。稳定化子的概念还可以推广到集合 S ，它的稳定化子定义为 $\text{Stab}_G(S)$ 。

概述

Schreier–Sims 算法主要试图解决这样一个问题：

- 给定大小为 n 的集合 Ω 上的一些置换组成的集合 S ，如何在计算机中高效地存储由 S 生成的置换群 $\langle S \rangle$ ，并完成一系列对该群的查询任务？

显然，这样的群 $\langle S \rangle$ 的规模可能很大，且远远大于集合 Ω 和生成集 S 的规模。比如， n 次对称群 S_n 的大小为 $n!$ ，但是它可以仅由两个置换生成。存储群中的每一个元素是不现实的。

类似于利用 [Gauss 消元法](#) 构建出向量空间的一组 [线性基](#)，Schreier–Sims 算法的思路是找到有限置换群 $\langle S \rangle$ 的一组「基」：

- 算法的输入是 S 的一个生成集，其中有若干个置换；
- 如果群 $\langle S \rangle$ 不平凡，总能找到在群 $\langle S \rangle$ 作用下位置会发生变化的点 α ，即 $\alpha \notin \text{Stab}_G(\alpha)$ ；
- 找到点 α 的轨道 αG ，并对轨道中的每个点 β ，都找到群 $\langle S \rangle$ 中一个置换 g ，它能够将点 α 移动到 β ；
- 找到点 α 的稳定化子 $\text{Stab}_G(\alpha)$ 的一个生成集；
- 递归地对 $\text{Stab}_G(\alpha)$ 调用该算法，直到得到平凡的群。

这个思路的合理性在于，点 α 的稳定化子 $\text{Stab}_G(\alpha)$ 是群 $\langle S \rangle$ 的子群，它的全体（右）陪集构成群 $\langle S \rangle$ 的分划，且这些陪集和点 α 的轨道 αG 一一对应，步骤 3 中求得的那些置换 g 就是这些陪集的代表元。这个陪集代表元的集合 T 称为稳定化子 $\text{Stab}_G(\alpha)$ 的 **陪集代表系** (transversal)。换句话说，群 $\langle S \rangle$ 中的每个元素 g 都和唯一的一对元素 (t, h) 对应，且 $g = th$ ，其中 $t \in T$ 且 $h \in \text{Stab}_G(\alpha)$ 。因而，只要能想办法存储子群 $\text{Stab}_G(\alpha)$ 和相应的陪集代表系 T ，就可以存储整个群 $\langle S \rangle$ 。然而，存储子群 $\text{Stab}_G(\alpha)$ 的问题也已经解决了——只要递归调用算法即可。

当然，算法的实现还有很多细节需要梳理，这是本文的主要内容。但是在那之前，首先要考察调用该算法之后得到的结果，看看算法将群存储为怎样的结构，能够解决怎样的查询问题。为此，应当明晰一些概念。

稳定化子链

假设对群 G 调用该算法，共进行了 k 次；且在第 k 次调用该算法时，输入是置换的集合 S_k ，找到的点是 α_k ，得到的陪集代表系是 $\{g_k\}$ ，得到的稳定化子的生成集是 S_{k+1} 。如果记 $S_0 = G$ 。那么，算法实际上得到了子群链

而且，每一个链中的子群都是一个稳定化子

因而，Schreier–Sims 算法可以看作就是在计算这样一个 **稳定化子链**（stabilizer chain）。

基和强生成集

如果集合 B 的子集 B' 满足，就称 B' 是置换群 G 的一组 **基**（base）。显然，上述算法得到了一组基 B 。这意味着，置换 σ 对这些点作用的结果在群 G 中唯一地确定了这个置换。而且，算法输出的这个基还满足条件

这就是说，基中的每个点都蕴含着关于群中的元素的有效信息。这样的基称为 **无冗余的**（nonredundant）。算法总是输出无冗余的基。这样的基对应的稳定化子链是严格递减的。

同时，算法得到的生成集的并集

也是群 G 的生成集，且成立 $\langle S \rangle = G$ 。满足这个条件的生成集称为群 G 相对于基 B 的 **强生成集**（strong generating set）。因而，Schreier–Sims 算法也可以看作是在计算群 G 的 **基和强生成集**（base and strong generating set, BSGS）。这个说法和稳定化子链的说法是等价的，本文不加以区分。

当然，算法还得到了一系列轨道 O_i 和相应的陪集代表系 $\{g_i\}$ 。这些轨道称为群 G 的 **基础轨道**（fundamental orbits）。当本文提及群 G 的稳定化子链或者基和强生成集时，总是默认相应的基础轨道和陪集代表系都已经一并求出。

数据结构

本文会提供一系列伪代码。伪代码中，群 G 的稳定化子链（或基和强生成集）存储在数据结构 $chain$ 中：

这个结构中的数据成员分别是当前群的生成集 S 、基础轨道 O 、相应的陪集代表系 $\{g\}$ 和存储为嵌套子结构的稳定化子 H 。当然，稳定化子 H 也是这样的一个结构。整个群实际存储在一个层状结构中，每层都描述了稳定化子链中的一个群。最内层是空的结构体，表示 $\{1\}$ 。

在伪代码中，该数据结构中的成员可以分别由 S 、 O 和 g 访问。轨道的首个元素默认是基中的点 α ，而相应的陪集代表元 g 默认是恒等变换。注意，虽然此处用数组下标访问了轨道和陪集代表系中的元素，但是它们未必存储为数组，而应当理解为它们提供了访问轨道首元素和根据轨道中的点查询相应的陪集代表元的方法。下文会讨论具体的实现细节。

应用

在获得群的基和强生成集后，能够解决一系列关于群的查询问题。其中，最基础的，也是算法竞赛中最常遇到的，是查询群的阶数和查询某个置换是否属于给定群的问题。

群的阶数

如果已知群 G 的基和强生成集，那么应用 Lagrange 定理和轨道稳定化子定理可知，群 G 的阶数可以计算为

所以，只要将所有陪集代表系的大小（或者等价地，基础轨道的长度）乘起来就可以得到群的阶数。

成员判定

已知群的基和强生成集，也可以判定某个置换是否属于群。这称为**成员判定**（membership testing）问题。

这个问题可以递归地解决。要判定，首先要找到包含的陪集的代表元。如果能够找到，那么设，就有且等价于；问题就转化为判定。如果找不到这样的，或者已经递归到了但是，就可以得出结论。其实，这个过程不仅判定了，而且在的情形下，还能够将表示为一系列陪集代表元的乘积，其中，。对于群中的元素，这样的表示存在且唯一。这再次证明了上面关于群的阶数的公式是正确的。

现在将该过程写成如下伪代码：

下文会看到，成员判定问题也是本文所讨论的 Schreier–Sims 算法的实现中的一个重要组成部分。

轨道、陪集代表系和稳定化子的计算

要实现 Schreier–Sims 算法，首先要解决如下子问题：[3](#)

- 给定群的生成集和一个点，如何求出轨道、相应的陪集代表系和稳定化子的生成集？

这就是本节要解决的问题。

轨道和陪集代表系的存储

要求得轨道和陪集代表系，只要直接搜索就好了。伪代码如下：

具体实现的时候，使用广度优先搜索和深度优先搜索都是可以的。搜索遍历到的状态数目是，只要能合理地存储轨道和陪集代表系，时间复杂度是完全可以接受的。

由于在置换群的语境下，轨道无非是至多个点的集合，为了高效完成查找和添加操作，可以使用布尔值数组或是无序集合存储。这样两个操作的时间复杂度都是的，整体的空间占用是的。当然，取决于陪集代表系的实现，可能还需要额外标记首元素的位置。

问题在于使用什么样的数据结构存储相应的陪集代表系。

直接存储

最简单的方法，当然是直接存储陪集代表系中的每一个元素。单个置换存储为[单行记号](#)，需要的空间恰为，所以存储这样的陪集代表系的空间复杂度是的。这样做的好处是访问单个陪集代表元的时间复杂度是的，代价是初次计算这些陪集代表元的时间复杂度是的。

Schreier 树

另外一种常见的做法是实现一个树形结构用于存储陪集代表系。它称为**Schreier 树**（Schreier tree）或**Schreier 向量**（Schreier vector）[4](#)。它以 1 为根，以轨道中的元素为顶点。每次在搜索过程中得到新的顶点时，就从旧的顶点到新的顶点连一条边，边上记录生成集中置换的序号（或指针）。因为已经存储了生成集，存储陪集代表系的额外空间复杂度是的。对于的规模很大的情形，这样做可以有效地节约空间，而且初次计算的时候复杂度是的。但是，副作用就是每次需要获得陪集代表元的时候，需要遍历顶点到根的路径上的边，重新计算陪集代表元，因而时间复杂度高度依赖于 Schreier 树的深度。对于一般的情形，树的深度可能达到级别。

在具体实现的时候，需要根据实际情况权衡算法的时空复杂度。对于算法竞赛可能涉及的情形，通常都不大，空间充

足，而时间复杂度常常成为瓶颈。稍后会看到，Schreier–Sims 算法最耗时的步骤恰好需要多次访问陪集代表元，因而使用直接存储的方式往往更优。但是对于某些应用场景，可能很大，存储空间可能更为紧张，就有可能需要使用 Schreier 树的方式存储陪集代表系。对于这种情况，为避免 Schreier 树深度过深，研究者提出了很多方法，可以在树的深度过深的时候重构出浅的 Schreier 树。有兴趣的读者可以参考文末的文献。

在伪代码中，本文不会区分具体的陪集代表系的实现，而只假设存储陪集代表系的数据结构实现了根据轨道元素访问和修改对应陪集代表元的操作。

Schreier 引理

在获得了轨道和陪集代表系后，Schreier 引理继而提供了获得稳定化子的生成集的方法。

▼ Schreier 引理

设群 G 有子群 H 。设 S 是子群 H 的（右）陪集代表系，且 $1 \in S$ ，并记 s 所在陪集的代表元为 h_s 。那么，集合

S 是子群 H 的一个生成集。它的元素称为子群 H 的 **Schreier 生成元**（Schreier generator）。

► 证明

因为陪集代表系 S 对应的子群就是稳定化子 H ，所以求出陪集代表系 S 后再结合群 G 的生成集 T 就能得到稳定化子 H 的生成集。

算法

只要对上面的伪代码稍作修改，就能在计算轨道和陪集代表系的同时得到相应的稳定化子的生成集：

伪代码中，对于每对 (g, s) ，只有轨道中不产生新的元素时， h_s 才是新的 Schreier 生成元；否则，它就是恒等变换。因而，算法中实际生成的 Schreier 生成元（包括最初的恒等变换）最多只有

$|T|$ 个。对于一般的情形，这个上界是紧的。⁶但是，对于实际要处理的有限群，这个上界相当地宽松：这些新得到的 Schreier 生成元大多数并都是之前得到的生成元的重复，或者可以由之前的生成元复合而成。

因为 Schreier–Sims 算法的基本流程可以实现为递归地调用上述计算轨道和稳定化子的算法，所以其实此时就已经得到了 Schreier–Sims 算法的一种朴素实现。但是，如果不加以筛选，Schreier 生成元的规模的快速增长速度是指数级的：反复应用 T 可知，最内层的稳定化子的生成集的规模将达到 $|T|^{n-1}$ 。这显然低效得荒诞，因为最内层的稳定化子是 $\{1\}$ 。

Sims 的工作提供了限制 Schreier 生成元的规模的快速增长速度的方法，它能够保证最终得到的强生成集 S 的大小是 $O(n^2)$ 的。这样就可以在多项式时间内计算基和强生成集。

Schreier–Sims 算法

为解决上述问题，本节讨论 Schreier–Sims 算法的一种增量实现，它得到的强生成集的大小是 $O(n^2)$ 的。

筛选

Schreier–Sims 算法对上述朴素算法的核心优化十分简明：它要求在向稳定化子的生成集添加任何 Schreier 生成元之前，都首先需要经过 **筛选**（sifting）。所谓筛选，就是首先判定新的 Schreier 生成元是否已经存在于已有的生成元生成的子群中，然后只添加那些尚不存在的生成元。为此，只需要使用前文的成员判定算法即可。

但是，能够这样做的前提是，基于当前群的已经产生了的 Schreier 生成元，早就构建好了它们生成的群的稳定化子链

（或基和强生成集）。这意味着，每次向稳定化子的生成集中添加一个新的 Schreier 生成元，都需要动态地维护内层的稳定化子链，以用于之后的筛选。但是，当前层每插入一个生成元，可能会产生多个 Schreier 生成元，也就可能会多次更新内层结构；而内层结构的每次更新，都可能会引发更内层结构的多次更新。

似乎之前提到的指数级爆炸的问题依然存在。其实不然。因为提前做好了筛选，只有待添加的生成元真的会引发某一层结构的扩大的时候，该层结构才会更新。这说明，单层结构更新的次数实际上等于该层结构存储的群严格增长的次数。但是，大小为 n 的群至多有长度为 n 的子群链；因为 Lagrange 定理保证，子群链长度每增加一，群的大小至少要翻倍。这就说明，单层结构至多只会更新 n 次，因而最后得到的强生成集的大小就是 2^n 的。

这个估计还可以进一步改进。因为此处出现的群 G 已知是 2 -次对称群 S_n 的子群，所以 G 的子群链长度不会超过 n 的子群链长度。可以证明^[7]， G 的严格递增子群链长度不会超过 n 。这说明，单层结构更新的次数其实是 n 的。显然，基的大小也不会超过 n 。故而，最后得到的强生成集的大小就是 2^n 的。

此处提到的筛选方法是 Sims 提出的，也称为 Sims 筛（Sims filter）。还有一种更为复杂的筛选方法，是由 Jerrum 提出的，也称为 Jerrum 筛（Jerrum filter），它能够保证得到的强生成集的大小是 2^n 的。有兴趣的读者可以自行学习。

对于筛选过程，有一个小优化是，在 S_n 的实现中，并不输出布尔值，而是输出最后得到的「筛渣」^[8]（即用 0 代替伪代码中的第 i 和第 j 行）。如果「筛渣」，就说明成员判定失败，此时可以直接将「筛渣」而不是原来的 G 添加到当前层。此处的「筛渣」已经除去了若干个陪集代表元的因子，因而移动了更少的元素，所以会减少局部的计算量。这个优化对于整体的复杂度没有任何影响。

过程

现在可以描述 Schreier–Sims 算法的具体过程：首先，初始化一个空结构，用于存储群的稳定化子链。然后，逐个向结构 S 中添加生成集 G 中的生成元，最后得到的结构 S 就是群 G 的稳定化子链。伪代码如下：

算法的核心在于向当前的 S 中添加新的生成元 这一步，即子程序 add 。正如前文所述，添加置换 σ 之前和之后，都需要保证 S 是稳定化子链。这样，在添加置换 σ 之前，可以首先做筛选。如果发现 σ 不在已有的群中，就 **增量地** 计算轨道、陪集代表元和 Schreier 生成元。此处的「增量」的含义是，已经计算过的，不要重复计算。这样才能保证正确的复杂度。

考虑如何将 add 改造为增量版本。算法搜索的状态空间是 2^n 。在添加新的生成元 σ 之后，状态空间将变成 2^n 。两者的差集就是

这意味着，当加入新的生成元 σ 的时候，首先需要计算新的生成元 σ 与旧的轨道和相应的陪集代表元的组合；如果在这个过程中还得到了新的轨道的元素，就再考虑这些元素与所有生成元（无论新旧）的组合；过程重复到轨道不再延长为止。

向结构 S 中添加置换 σ 的伪代码如下：

这样就得到了完整的 Schreier–Sims 算法。

另一种实现

上述的实现已经是正确的，但是 add 行和 add 行略显重复。基于此，Knuth 在论文中提出了一种递归实现，更为简明。他的做法是，将这个重复的部分视作是对陪集剩余系（和轨道）的更新。每次更新陪集剩余系都要和所有的生成元组合，根据是否产生了新的陪集代表元，决定是递归地调用自身还是添加生成元的程序。伪代码如下：

将此处的伪代码和上节的相比，就可以知道它是正确的。而且，两者复杂度并无差异。

复杂度

为了分析 Schreier–Sims 算法的复杂度，需要一些记号。设置换的长度为 ℓ ，生成集的大小为 n 。得到的（无冗余）基的长度记为 k 。而且，最后得到的自外向内第 i 层的稳定化子 H_i 中，生成元的数目记为 n_i ，陪集代表系的大小（或轨道的长度）记为 $|H_i|$ 。下面分析利用上述 Schreier–Sims 算法的增量实现所需要的时间复杂度。算法主要分为两部分：筛选，以及对轨道、陪集代表系和 Schreider 生成元的计算。

最初输入的生成元和算法中得到的 Schreider 生成元都需要进行筛选，因而筛选过程执行的总次数是 n 。单次筛选需要与 n 个陪集代表元计算置换乘积。设计算与单个陪集代表元的乘积的时间是 $O(\ell)$ 。直接存储陪集代表元时， $O(n^2)$ ；而用 Schreier 树存储陪集代表元时， $O(n)$ 。执行筛选过程的时间复杂度总共为 $O(n^2)$ 。

对于轨道等信息的计算，因为是增量实现，状态空间中的每对 (i, j) 都只计算了至多一次。对于轨道和陪集代表系的计算，根据存储方式不同，单次计算陪集代表元的时间复杂度可能是 $O(\ell)$ 的或是 $O(n)$ 的。但是，无论如何，这都不超过计算 Schreider 生成元的复杂度。直接存储时，它是 $O(n^2)$ 的；使用 Schreier 树时，它是 $O(n)$ 的。与筛选过程的总时间复杂度比较，会发现计算这些信息的时间复杂度都不会超过筛选需要的时间复杂度。所以，Schreier–Sims 算法的时间复杂度就是上一段得到的 $O(n^2)$ 。

至于空间复杂度，算法最后得到的数据结构中存储了 n 个生成元和 n 个陪集代表元。如果使用直接存储，生成元和陪集代表元都需要 $O(n)$ 的空间；如果使用 Schreiner 树，生成元需要 $O(n)$ 的空间，而陪集代表元只需要 $O(1)$ 的空间。

前文已经说明， n 次对称群中严格递增子群链的长度是 $O(\log n)$ 的，这对 n 和 ℓ 都适用。因而，使用直接存储陪集代表系的方式实现的 Schreier–Sims 算法，时间复杂度是 $O(n^2)$ 的，空间复杂度是 $O(n)$ 的。当然对于 n 的情形，一个更好的估计是时间复杂度⁹和空间复杂度 $O(n)$ 。对于随机的生成集的情形，实际测试发现算法的复杂度明显低于 $O(n^2)$ ，而大致是 $O(n)$ 的。

虽然相较于直接存储，用 Schreiner 树会在时间复杂度中引入额外的 $O(\ell)$ 的指数，但对于 ℓ 很大，但是群本身远小于 n 次对称群的情形，它的空间复杂度是 $O(n)$ 的，远小于直接存储的 $O(n^2)$ 。但是在算法竞赛中，很难遇到这样使用 Schreiner 树存储更优的情形。

参考实现

此处提供一个 Schreier–Sims 算法的参考实现。因为规模较小，实现中直接指定基而不是通过算法选择它。这样做的好处是，自内向外第 i 层（不计空结构体）的群中的置换只就会改变前 n_i 个元素，方便后续计算。代码中的另一项优化是，在存储陪集代表元的时候，存储的实际上是它的逆置换，这简化了置换的运算。

► 参考实现

习题

- [LOJ 177. 生成子群阶数](#)
- [\[WC2017\] 棋盘](#)
- [Permutations](#)
- [\[Grand Prix of Yekaterinburg 2015\] Problem H. Heimdall](#)

参考资料与注释

- [Schreier–Sims algorithm - Wikipedia](#)
- [Sims, Charles C. Computational methods in the study of permutation groups. Computational Problems in Abstract Algebra, pp. 169–183, Pergamon, Oxford, 1970.](#)
- [Knuth, Donald E. Efficient representation of perm groups. Combinatorica 11 \(1991\), no. 1, 33–43.](#)
- [Ákos Seress, Permutation Group Algorithms, Cambridge University Press](#)
- [Alexander Hulpke's Notes on Computational Group Theory](#)
- [Derek Holt's Slides on The Schreier–Sims algorithm for finite permutation groups](#)
- [Martin Jaggi, Implementations of 3 Types of the Schreier–Sims Algorithm, MAS334 - Mathematics Computing Project, 2005](#)

- [Henrik Bäärnhielm. The Schreier–Sims algorithm for matrix groups](#)
-

1. Knuth 的论文是在 1991 年发表的，但是他的改进在 1981 年就通过会议广泛地宣传。论文是基于他的会议讲稿写作的。 [↩](#)
 2. 不要与 Monto Carlo 方法混淆。此处的 Monte Carlo 算法是指出错概率恒定且任意小的随机算法。 [↩](#)
 3. 这个问题以及本节的算法都并不需要假设所讨论的群作用是置换作用，因而可以应用于更广泛的场景。比如，如果将这些算法应用于共轭作用，同样可以求得轨道（共轭类）、陪集代表系和稳定化子（中心化子）。 [↩](#)
 4. 因为这个树形结构可以通过一系列指向父节点的指针来实现，所以也称作 Schreier 向量。 [↩](#)
 5. 这个 的条件对于 Schreier 引理的成立不是必要的。 [↩](#)
 6. [Nielsen–Schreier 定理](#) 说明，对于由 n 个生成元生成的 [自由群](#)，它的指数为 m 的子群是由 nm 个生成元生成的自由群。 [↩](#)
 7. 参见 [Cameron, P. J., Solomon, R., & Turull, A. \(1989\). Chains of subgroups in symmetric groups. Journal of algebra, 127\(2\), 340-352.](#) [↩](#)
 8. 这并不是什么严格的术语，在不同的英文文献中可能称作 sifted 或者 sifted element。 [↩](#)
 9. Knuth 的论文给出的上界还要再少一个对数因子，这需要对群的稳定化子链的基础轨道长度做更仔细的估计。 [↩](#)
-

本页面最近更新：2024/11/28 02:15:12, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [c-forrest](#), [Enter-tainer](#), [iamtwz](#), [lr1d](#), [isdanni](#), [ksyx](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用